

Test Design Document

Campus Equipment Booking & Return System

Course: Software Testing and Validation - Addis Ababa University **Group members:** (add names and student IDs here)

This document derives test cases from the requirements using four formal techniques: equivalence partitioning, boundary value analysis, a decision table, and state transition testing. Every case maps to a real automated test in the repository, so this document cannot drift from the code.

1. Requirements under test

Req ID	Requirement
REQ-DATES	A booking's start date is required, its end date is required, and the end date must be strictly after the start date.
REQ-QTY	Requested quantity must be a positive integer, and must not exceed either the equipment's total quantity or the quantity still available for the requested period.
REQ-MAINT	Equipment under maintenance cannot be booked.
REQ-APPROVE	Only an admin may approve/reject a REQUESTED booking; approval re-checks availability against all <i>other</i> bookings for the same period.
REQ-ACTIVATE	Only an admin may hand over (activate) an APPROVED booking, which decrements the equipment's physical available-quantity counter; this fails if the equipment went into maintenance or the physical stock is insufficient since approval.
REQ-RETURN	Only ACTIVE or LATE bookings can be returned; returning restores the physical available-quantity counter, capped at the equipment's total.
REQ-LATE	An ACTIVE booking becomes LATE once the current time passes its expected return date.
REQ-CANCEL	Only the booking's owner or an admin may cancel a REQUESTED or APPROVED booking, and only before its start date (the cancellation deadline).
REQ-STATE	Booking status only moves along the transitions in section 4.
REQ-AUTH	Registration requires a password of at least 8 characters and a unique username; admin-only routes return 403 for non-admins.

2. Equivalence Partitioning (EP)

2.1 Booking dates (REQ-DATES)

Class	Description	Valid?	Automated test
EP-1	start_date missing (None)	Invalid	test_ep001_missing_start_date_raises
EP-2	end_date missing (None)	Invalid	test_ep002_missing_end_date_raises
EP-3	end == start	Invalid	test_ep003_start_equal_to_end_raises
EP-4	end before start	Invalid	test_ep004_start_after_end_raises
EP-5	end after start	Valid	test_ep005_end_after_start_is_accepted

2.2 Quantity (REQ-QTY)

Class	Description	Valid?	Automated test
-------	-------------	--------	----------------

EP-6	quantity is None	Invalid	test_ep006_none_quantity_raises
EP-7	quantity == 0	Invalid	test_ep007_zero_quantity_raises
EP-8	quantity negative	Invalid	test_ep008_negative_quantity_raises
EP-9	quantity positive	Valid	test_ep009_positive_quantity_is_accepted

2.3 Equipment bookability (REQ-MAINT)

Class	Description	Valid?	Automated test
EP-10	maintenance = False, sufficient stock	Bookable	test_ut006_check_availability_true_when_stock_sufficient
EP-11	maintenance = True	Not bookable	test_ut008_check_availability_raises_under_maintenance
EP-12	quantity requested > equipment.quantity	Not bookable	test_ut007_check_availability_raises_when_quantity_exceeds_stock
EP-13	quantity requested > remaining reserved-adjusted stock	Not bookable	test_ut009_check_availability_raises_when_reserved_by_others

3. Boundary Value Analysis (BVA)

3.1 Quantity lower boundary

Test case	Value	Expected	Automated test
BVA-1	0	Rejected	test_ep007_zero_quantity_raises
BVA-2	1 (minimum valid)	Accepted	test_bva001_quantity_of_one_is_accepted

3.2 Date-overlap boundary (touching windows)

Test case	Scenario	Expected	Automated test
ST-002	Window B starts exactly when window A ends	No overlap (half-open interval)	test_st003_adjacent_windows_do_not_overlap
ST-004	Window B starts 1 unit before A ends	Overlap	test_st004_partial_overlap_is_detected

3.3 Capacity boundary (the exact scenario DEF-002 broke)

Test case	Scenario	Expected	Automated test
BVA-3	Booking requests exactly the remaining capacity (own quantity == total, no other bookings)	Approval succeeds	test_ut013_approve_requested_booking_succeeds, test_it305_full_lifecycle_approve_activate_return
BVA-4	A second booking requests 1 more unit than remains after another CONFIRMED booking	Rejected	test_ut015_approve_rechecks_availability_and_rejects_conflict, test_it304_second_conflicting_booking_rejected

BVA-3 is the exact boundary DEF-002 got wrong (see the Defect Log): before the fix, a booking requesting *all* of the remaining capacity was incorrectly rejected at approval time because it was double-counted against itself. This is a real example of why the boundary - not just the “clearly valid, clearly invalid” middle cases - has to be tested explicitly.

4. Decision Table Testing - Booking Approval

Conditions: - C1: Actor is an admin? - C2: Booking is currently REQUESTED? - C3: Equipment is not under maintenance? - C4: Requested quantity is available after excluding this booking's own reservation?

Rule	C1	C2	C3	C4	Action	Automated test
DT-1	N	-	-	-	Rejected (403, route-level)	test_it107_admin_only_route_forbidden_for_student
DT-2	Y	N	-	-	Rejected: "Only requested bookings..."	test_ut014_approve_non_requested_booking_raises
DT-3	Y	Y	N	-	Rejected: maintenance (caught at creation, not reachable at approval since a REQUESTED booking can't exist for maintenance equipment)	test_ut008_check_availability_raises_under_maintenance
DT-4	Y	Y	Y	N	Rejected: availability conflict	test_ut015_approve_rechecks_availability_and_rejects_conflict
DT-5	Y	Y	Y	Y	Approved	test_ut013_approve_requested_booking_succeeds

A second decision table for **activation** (the equivalent step for REQ-ACTIVATE), since it has its own independent condition set:

Conditions: C1 admin?, C2 booking APPROVED?, C3 equipment not under maintenance (may have changed since approval)?, C4 physical stock >= requested quantity?

Rule	C1	C2	C3	C4	Action	Automated test
DT-6	Y	N	-	-	Rejected: "Only approved bookings..."	test_ut018_activate_non_approved_booking_raises
DT-7	Y	Y	N	-	Rejected: maintenance	test_ut019_activate_raises_when_equipment_under_maintenance
DT-8	Y	Y	Y	N	Rejected: "Not enough physical equipment..."	test_ut020_activate_raises_when_physical_stock_insufficient
DT-9	Y	Y	Y	Y	Activated, stock decremented	test_ut017_activate_approved_booking_reduces_available_quantity

5. State Transition Testing

5.1 State model

```
REQUESTED --approve--> APPROVED --activate--> ACTIVE --time passes--> LATE --return--> RETURNED
REQUESTED --reject--> REJECTED
REQUESTED --cancel--> CANCELLED
APPROVED --cancel--> CANCELLED
ACTIVE --return--> RETURNED
```

REJECTED, RETURNED, and CANCELLED are terminal. Implemented as data (VALID_TRANSITIONS) in app/services/booking_service.py, not scattered conditionals, so this document and the code cannot drift apart. CANCELLED was added to this table during the refinement pass described in the Defect Log - it was entirely absent beforehand despite being required.

5.2 Valid transitions

From	To	Automated test
REQUESTED	APPROVED	test_st2xx_transition_booking_applies_valid_transition[REQUESTED-APPROVED]
REQUESTED	REJECTED	same, parametrized
REQUESTED	CANCELLED	same, parametrized; also test_it307_owner_can_cancel_own_booking
APPROVED	ACTIVE	same, parametrized; also test_it305_full_lifecycle_approve_activate_return
APPROVED	CANCELLED	same, parametrized; also test_it309_admin_can_cancel_a_students_booking
ACTIVE	LATE	test_ut021_mark_late_transitions_active_past_due_booking, test_it310_my_bookings_auto_flags_late
ACTIVE	RETURNED	test_it305_full_lifecycle_approve_activate_return
LATE	RETURNED	test_ut025_return_late_booking_succeeds

5.3 Invalid transitions (must be rejected)

From	To	Automated test
RETURNED	APPROVED	test_st3xx_transition_booking_rejects_invalid_transition[RETURNED-APPROVED]
REJECTED	ACTIVE	same, parametrized
CANCELLED	APPROVED	same, parametrized
RETURNED	CANCELLED	same, parametrized
REQUESTED	ACTIVE (skip approval)	same, parametrized - this is the exact transition DEF (state-machine) risk category targets; not currently reachable via any route, verified structurally
REQUESTED	RETURNED	same, parametrized
CANCELLED	ACTIVE	same, parametrized
LATE	APPROVED	same, parametrized
APPROVED	REQUESTED (backwards)	same, parametrized
ACTIVE	APPROVED (backwards)	same, parametrized

E2E-011 additionally proves CANCELLED really is terminal against the live running application: it cancels a booking through the UI, confirms the Cancel button disappears (client-side), then forges a second cancel request directly against the route and confirms the backend still rejects it and the status is unchanged - not just that the button was hidden.

6. Automation mapping summary

Technique	Unit	Integration	E2E
Equivalence Partitioning	test_validation.py	-	-
Boundary Value Analysis	test_validation.py, test_booking_service.py	test_it304, test_it305	-
Decision Table	test_booking_service.py (UT-013 to UT-020)	test_it107, test_it305, test_it306	E2E-006, E2E-010
State Transition	test_state_machine.py	test_it305, test_it307 to test_it310	E2E-008, E2E-010, E2E-011

Every case in this document maps to a real, currently-passing automated test (unit and integration levels; E2E level written and structurally verified but not yet executed against a real browser - see the Test Summary Report).